

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	SOAP Web services				
NÚMERO DE LA PRÁCTICA:	7	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	02/06/2026	HORA DE PRESENTACIÓN:	21:50		
INTEGRANTE(s): - Barrios Medina, Mathias Alonso - Huacani Jara Denise Andrea - Hanco Mullisaca, Sergio Danilo - Pacheco Palo, Fabiana Francinet - Quispe Condori, Alvaro Raul				NOTA:	
DOCENTE: Mg. Maribel Molina Barriga					

SOLUCIÓN Y RESULTADOS
I. DESARROLLO DE EJERCICIOS RESUELTOS POR EL DOCENTE Link del github: https://github.com/unsa-semester-2026-A/sis_dis_lab.git Programa Aplicativo: Servicio SOAP de Suma En esta sección se presenta la solución corregida del servicio SOAP de suma. La estructura original no compilaba debido a la ausencia de una interfaz que actúe como Service Endpoint Interface (SEI) para la generación del proxy dinámico en el lado del cliente. Hemos corregido esta arquitectura separando la interfaz y su clase de implementación, garantizando que compile y funcione correctamente. Para cada ejercicio se incluye el flujo de funcionamiento que explica la relacion entre cada componentes y explicación- importancia del código propuesto. A. Flujo de Funcionamiento del Servicio de Suma El flujo de llamadas y ciclo de vida de este servicio web se describe secuencialmente a continuación: Definición de Contrato (Interfaz): Se crea la interfaz CalculadoraSOAP anotada con @WebService y sus métodos expuestos con @WebMethod. Esta define los tipos y operaciones que compondrán el WSDL. Implementación de Lógica (Servicio): La clase CalculadoraSOAPImpl implementa CalculadoraSOAP. Mediante la anotación @WebService(endpointInterface = "...", serviceName = "CalculadoraSOAPServicio") se asocia la lógica al contrato formal y se fuerza el nombre del servicio generado. Publicación del Endpoint: El programa Publicador invoca a Endpoint.publish("http://localhost:8080/calculadora", new CalculadoraSOAPImpl()). Esto inicia un servidor HTTP ligero incorporado en la JVM que

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

escucha en el puerto 8080 y autogenera el esquema descriptivo XML (WSDL) consultable en `http://localhost:8080/calculadora?wsdl`, ya se encuentra disponible y espera peticiones.

4. **Consumo del Cliente:** El programa ClienteSOAP consumirá el servicio y muestra el resultado, accede al WSDL, crea una instancia de Service y obtiene la interfaz de comunicación (Proxy) a través de `service.getPort()`. Al llamar a `calc.sumar(10, 20)`, se realiza una llamada remota real: los parámetros se serializan en un sobre XML SOAP, viajan por HTTP, se procesan en el servidor, este responde con un XML y el cliente lo deserializa a un entero imprimible.

Contrato del Servicio (CalculadoraSOAP.java):

```
package Ej_Resuelto;

import javax.ws.WebMethod;
import javax.ws.WebService;

// Interface que define el contrato del servicio SOAP de calculadora.
@WebService
public interface CalculadoraSOAP {
    @WebMethod
    /* Permite que el metodo aparezca en el contrato WSDL y
    pueda ser invocado por clientes externos via SOAP.
    */
    public int sumar(int a, int b);
}
```

Implementación del Servicio (CalculadoraSOAPImpl.java):

```
package Ej_Resuelto;

import javax.ws.WebService;

// Clase que implementa el servicio SOAP de calculadora.
// Se asocia con la interface CalculadoraSOAP para definir el contrato.
@WebService(endpointInterface = "Ej_Resuelto.CalculadoraSOAP", serviceName = "CalculadoraSOAPService")
public class CalculadoraSOAPImpl implements CalculadoraSOAP {

    @Override
    public int sumar(int a, int b) {
        return a + b;
    }
}
```

Servidor Publicador (Publicador.java):

```
package Ej_Resuelto;

//Paso 2: Publicar el Servicio
import javax.xml.ws.Endpoint;
import Ej_Resuelto.CalculadoraSOAPImpl; //Esto se agrego
public class Publicador{
    public static void main(String[] args){
        Endpoint.publish( //levanta el servicio en una URL especifica
            "http://localhost:8080/calculadora",
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```

        new CalculadoraSOAPImpl()
    );
    System.out.println("Servicio SOAP activo");
}
}
//Aquí el servicio ya esta corriendo y esperando clientes

```

Cliente Consumidor (ClienteSOAP.java):

```

package Ej_Resuelto;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import Ej_Resuelto.CalculadoraSOAP;

// Paso 3: Consumidor Java
public class ClienteSOAP {
    public static void main(String[] args) throws Exception {
        // Direccion URL del WSDL del servicio web
        URL url = new URL("http://localhost:8080/calculadora?wsdl");

        // QName identifica el servicio dentro del WSDL (namespace + nombre local)
        QName qname = new QName("http://Ej_Resuelto/", "CalculadoraSOAPService");

        // Creamos la instancia del servicio
        Service service = Service.create(url, qname);

        // Obtenemos el puerto de acceso (proxy) del servicio
        CalculadoraSOAP calc = service.getPort(Ej_Resuelto.CalculadoraSOAP.class);

        // Llamada al metodo remoto
        System.out.println("Resultado de la suma remota (10 + 20): " + calc.sumar(10,20));
    }
}

```

B. ANALISIS DEL EJERCICIO DADO

Se observa lo siguiente:

1. Cuando se ejecuta el Publicador, se inicia un servidor HTTP embebido en mi máquina que expone el servicio SOAP. En ese momento, JAX-WS genera automáticamente un contrato WSDL en formato XML. Este documento describe el servicio, sus operaciones disponibles y los tipos de datos que se intercambian. El WSDL funciona como un contrato estandarizado que permite a cualquier cliente conocer cómo interactuar con el servicio.
2. Al ejecutar el ClienteSOAP, no se invoca directamente el método Java, sino que se construye un mensaje SOAP en XML. Este mensaje se envía mediante HTTP al servidor, que lo interpreta, ejecuta la lógica definida en la clase (por ejemplo, la suma de dos números) y devuelve una respuesta también en XML. El cliente procesa esa respuesta y muestra el resultado en consola. Lo que parece una simple llamada a un método es, en realidad, un intercambio de documentos XML estructurados.
3. En cuanto a la latencia, al trabajar en localhost es prácticamente imperceptible, ya que el mensaje SOAP no viaja por la red externa. Sin embargo, en un entorno real la latencia puede verse afectada por tres factores principales:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

el tiempo de transmisión en la red, el procesamiento necesario para interpretar y ejecutar la lógica del servicio, y el tamaño de los mensajes SOAP (el XML es más pesado que otros formatos como JSON).

Compilación y Ejecución:

1. Se abre una terminal de comandos (como PowerShell) en la raíz del proyecto.
2. Compilación de todos los archivos del paquete Ej_Resuelto ejecutando:

```
javac -d . codigo_compartir/Ej_Resuelto/*.java
```

3. Se inicia el servidor que publica el servicio web:

```
java Ej_Resuelto.Publicador
```

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> javac -d . codigo_compartir/Ej_Resuelto/*.java
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_Resuelto.Publicador
o Servicio SOAP activo
█
```

4. En otra ventana o terminal de comandos, se ejecuta el programa cliente para consumir el servicio:

```
java Ej_Resuelto.ClienteSOAP
```

5. Se verifica que la consola del cliente imprima: Resultado de la suma remota (10 + 20): 30

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_Resuelto.ClienteSOAP
Resultado de la suma remota (10 + 20): 30
█
```

II. DESARROLLO DE EJERCICIOS PROPUESTOS

A continuación, se detalla la resolución de los problemas planteados en la guía de práctica.

EJERCICIO 1: Servicio SOAP - Conversor de Temperatura y Servicio de Ventas en Línea

Este ejercicio abarca el desarrollo de un conversor bidireccional de temperatura y el diseño de implementación de un servicio SOAP básico para gestionar ventas de productos en línea.

A. Flujo de Funcionamiento del Conversor

Se documenta todo el proceso y explicación por cada parte de ambos problemas.

• Conversor de Temperatura:

1. La interfaz ConversorSOAP define los contratos para cToF (Celsius a Fahrenheit) y fToC (Fahrenheit a Celsius).
2. ConversorSOAPImpl implementa las fórmulas aritméticas correspondientes.
3. El publicador arranca el servicio en `http://localhost:8081/conversor`.
4. El cliente se conecta y realiza la invocación. El servidor recibe los valores en un sobre XML (`<arg0>30.0</arg0>`), efectúa la operación y devuelve un sobre SOAP de respuesta que contiene el resultado procesado en punto flotante (`double`).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

Contrato del Conversor (ConversorSOAP.java):

```
package Ej_propuestos.ej_01;

import javax.ws.WebMethod;
import javax.ws.WebService;

// Interface que define el contrato del servicio de conversion de temperatura.
@WebService(targetNamespace = "http://conversor.soap/")
public interface ConversorSOAP {

    @WebMethod
    public double cToF(double c);

    @WebMethod
    public double fToC(double f);
}
```

Implementación del Conversor (ConversorSOAPImpl.java):

```
package Ej_propuestos.ej_01;

import javax.ws.WebService;

// Clase que implementa el servicio SOAP de conversion de temperatura.
@WebService(
    endpointInterface = "Ej_propuestos.ej_01.ConversorSOAP",
    serviceName = "ConversorSOAPService",
    targetNamespace = "http://conversor.soap/"
)
public class ConversorSOAPImpl implements ConversorSOAP {

    @Override
    public double cToF(double c) {
        // Formula para Celsius a Fahrenheit: (C * 9/5) + 32
        return (c * 9.0 / 5.0) + 32.0;
    }

    @Override
    public double fToC(double f) {
        // Formula para Fahrenheit a Celsius: (F - 32) * 5/9
        return (f - 32.0) * 5.0 / 9.0;
    }
}
```

Publicador del Conversor (Publicador.java):

```
package Ej_propuestos.ej_01;

import javax.xml.ws.Endpoint;

// Clase para publicar el servicio SOAP en una URL especifica.
public class Publicador {
    public static void main(String[] args) {
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```
// Se define el puerto 8081 para evitar conflictos con el puerto 8080
String url = "http://localhost:8081/conversor";
Endpoint.publish(url, new ConversorSOAPImpl());
System.out.println("Servicio SOAP de Conversor de Temperatura activo en: " + url);
}
}
```

Cliente del Conversor (ClienteSOAP.java):

```
package Ej_propuestos.ej_01;

import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

// Cliente Java que consume el servicio de conversion de temperatura de forma remota.
public class ClienteSOAP {
    public static void main(String[] args) throws Exception {
        // URL de definicion del servicio (WSDL)
        URL url = new URL("http://localhost:8081/conversor?wsdl");

        // QName identifica al servicio (namespace y local name)
        QName qname = new QName("http://conversor.soap/", "ConversorSOAPService");

        // Creacion del servicio
        Service service = Service.create(url, qname);

        // Obtencion del puerto utilizando la interface
        ConversorSOAP conversor = service.getPort(ConversorSOAP.class);

        // Pruebas de conversion
        double c = 30.0;
        double f = conversor.cToF(c);
        System.out.println("Llamando a cToF(" + c + ")... Resultado: " + f + " Fahrenheit");

        double f2 = 86.0;
        double c2 = conversor.fToC(f2);
        System.out.println("Llamando a fToC(" + f2 + ")... Resultado: " + c2 + " Celsius");
    }
}
```

Compilación y Ejecución - Convertidor de Temperatura

1. Se abre un terminal y se compila todas las clases de este ejercicio:

```
javac -d . codigo_compartir/Ej_propuestos/ej_01/*.java
```

2. Se ejecuta el servidor:

```
java Ej_propuestos.ej_01.Publicador
```

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> javac -d . codigo_compartir/Ej_propuestos/ej_01/*.java
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_propuestos.ej_01.Publicador
Servicio SOAP de Conversor de Temperatura activo en: http://localhost:8081/conversor
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

3. En otros terminales, se ejecuta los respectivos programas cliente para realizar las pruebas:

```
java Ej_propuestos.ej_01.ClienteSOAP
```

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_propuestos.ej_01.ClienteSOAP
Llamando a cToF(30.0)... Resultado: 86.0 Fahrenheit
Llamando a fToC(86.0)... Resultado: 30.0 Celsius
```

B. Flujo de Funcionamiento del Sistema de Ventas en Línea

• Servicio de Ventas en Línea:

1. Se define la clase Producto que actúa como un JavaBean serializable (con atributos ID, nombre, precio y stock, y constructor vacío).
2. El servicio expone métodos clave: obtenerProductos() (listar catálogo), buscarProducto(id) (detalles individuales) y realizarVenta(id, cantidad).
3. La lógica interna del servicio mantiene el estado del inventario en memoria. Al invocar realizarVenta, el servidor evalúa la disponibilidad del stock. Si es suficiente, deduce las unidades vendidas, calcula el costo total y retorna un mensaje de éxito; si es insuficiente, aborta la operación y responde con un mensaje de error.

Modelo de Producto (Producto.java):

```
package Ej_propuestos.ej_01;

import java.io.Serializable;

// Clase que representa un producto.
// Debe ser serializable e implementar un constructor vacio para ser transmitida en SOAP.
public class Producto implements Serializable {
    private static final long serialVersionUID = 1L;

    private int id;
    private String nombre;
    private double precio;
    private int stock;

    // Constructor vacio obligatorio para JAX-WS
    public Producto() {}

    public Producto(int id, String nombre, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public double getPrecio() {
    return precio;
}

public void setPrecio(double precio) {
    this.precio = precio;
}

public int getStock() {
    return stock;
}

public void setStock(int stock) {
    this.stock = stock;
}
}

```

Contrato del Servicio de Ventas (VentasSOAP.java):

```

package Ej_propuestos.ej_01;

import javax.ws.WebMethod;
import javax.ws.WebService;

// Interface del servicio SOAP para ventas en linea.
@WebService(targetNamespace = "http://ventas.soap/")
public interface VentasSOAP {

    @WebMethod
    public Producto[] obtenerProductos();

    @WebMethod
    public Producto buscarProducto(int id);

    @WebMethod
    public String realizarVenta(int idProducto, int cantidad);
}

```

Implementación de Ventas (VentasSOAPImpl.java):

```

package Ej_propuestos.ej_01;

import javax.ws.WebService;
import java.util.ArrayList;
import java.util.List;

```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```
// Clase que implementa el servicio SOAP de ventas en linea.
@WebService(
    endpointInterface = "Ej_propuestos.ej_01.VentasSOAP",
    serviceName = "VentasSOAPService",
    targetNamespace = "http://ventas.soap/"
)
public class VentasSOAPImpl implements VentasSOAP {
    private List<Producto> inventario;

    public VentasSOAPImpl() {
        inventario = new ArrayList<Producto>();
        inventario.add(new Producto(101, "Laptop Gamer", 1200.0, 5));
        inventario.add(new Producto(102, "Teclado Mecanico", 85.5, 20));
        inventario.add(new Producto(103, "Mouse Optico", 35.0, 15));
    }

    @Override
    public Producto[] obtenerProductos() {
        return inventario.toArray(new Producto[0]);
    }

    @Override
    public Producto buscarProducto(int id) {
        for (Producto p : inventario) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }

    @Override
    public String realizarVenta(int idProducto, int cantidad) {
        for (Producto p : inventario) {
            if (p.getId() == idProducto) {
                if (p.getStock() >= cantidad) {
                    p.setStock(p.getStock() - cantidad);
                    double total = p.getPrecio() * cantidad;
                    return "Venta exitosa. ID Producto: " + idProducto + ". Cantidad: " + cantidad +
                        ". Total cobrado: $" + total + ". Stock actual: " + p.getStock();
                } else {
                    return "Error: Stock insuficiente. Unidades disponibles: " + p.getStock();
                }
            }
        }
        return "Error: Producto con ID " + idProducto + " no existe.";
    }
}
```

Publicador del Servicio de Ventas (PublicadorVentas.java):

```
package Ej_propuestos.ej_01;

import javax.xml.ws.Endpoint;
```

```
// Clase para publicar el servicio SOAP de ventas en linea.
public class PublicadorVentas {
    public static void main(String[] args) {
        // Se define el puerto 8082 para el servicio de ventas
        String url = "http://localhost:8082/ventas";
        Endpoint.publish(url, new VentasSOAPImpl());
        System.out.println("Servicio SOAP de Ventas activo en: " + url);
    }
}
```

Cliente de Ventas (ClienteVentas.java):

```
package Ej_propuestos.ej_01;

import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

// Cliente Java que consume el servicio de ventas en linea remotamente.
public class ClienteVentas {
    public static void main(String[] args) throws Exception {
        // URL de definicion del servicio (WSDL)
        URL url = new URL("http://localhost:8082/ventas?wsdl");

        // QName identifica al servicio (namespace y local name)
        QName qname = new QName("http://ventas.soap/", "VentasSOAPService");

        // Creacion del servicio
        Service service = Service.create(url, qname);

        // Obtencion del puerto utilizando la interface
        VentasSOAP ventas = service.getPort(VentasSOAP.class);

        // 1. Mostrar catalogo de productos
        System.out.println("=== CATALOGO DE PRODUCTOS ===");
        Producto[] lista = ventas.obtenerProductos();
        for (Producto p : lista) {
            System.out.println("ID: " + p.getId() +
                " | Nombre: " + p.getNombre() +
                " | Precio: $" + p.getPrecio() +
                " | Stock: " + p.getStock());
        }
        System.out.println("=====");

        // 2. Intentar realizar una venta exitosa
        int idCompra = 102;
        int cantidadCompra = 3;
        System.out.println("\nComprando " + cantidadCompra + " unidades del producto ID: " + idCompra +
            "...");
        String resultado1 = ventas.realizarVenta(idCompra, cantidadCompra);
        System.out.println("Respuesta del servidor: " + resultado1);

        // 3. Intentar realizar una venta con stock insuficiente
        int idCompra2 = 101;
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```

        int cantidadCompra2 = 10;
        System.out.println("\nComprando " + cantidadCompra2 + " unidades del producto ID: " + idCompra2
+ "...");
        String resultado2 = ventas.realizarVenta(idCompra2, cantidadCompra2);
        System.out.println("Respuesta del servidor: " + resultado2);
    }
}

```

Compilación y Ejecución - Ventas

1. Se abre un terminal y se compila todas las clases de este ejercicio:

```
javac -d . codigo_compartir/Ej_propuestos/ej_01/*.java
```

2. Se ejecuta el servidor:

```
java Ej_propuestos.ej_01.PublicadorVentas
```

```

PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_propuestos.ej_01.PublicadorVentas
Servicio SOAP de Ventas activo en: http://localhost:8082/ventas

```

3. En otros terminales, se ejecuta los respectivos programas cliente para realizar las pruebas:

```
java Ej_propuestos.ej_01.ClienteVentas
```

```

PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_propuestos.ej_01.ClienteVentas
=== CATALOGO DE PRODUCTOS ===
ID: 101 | Nombre: Laptop Gamer | Precio: $1200.0 | Stock: 5
ID: 102 | Nombre: Teclado Mecanico | Precio: $85.5 | Stock: 20
ID: 103 | Nombre: Mouse Optico | Precio: $35.0 | Stock: 15
=====

Comprando 3 unidades del producto ID: 102...
Respuesta del servidor: Venta exitosa. ID Producto: 102. Cantidad: 3. Total cobrado: $256.5. Stock actual: 17

Comprando 10 unidades del producto ID: 101...
Respuesta del servidor: Error: Stock insuficiente. Unidades disponibles: 5

```

C. MEJORAS REALIZADAS:

• Mejora: Interfaces Gráficas de Usuario en Swing para Ejercicio 1

Para mejorar y facilitar la interacción con los servicios SOAP de conversión de temperatura y de ventas, se diseñaron e implementaron dos interfaces gráficas de escritorio utilizando la biblioteca estándar `javax.swing`. Ambas interfaces operan de forma asíncrona mediante hilos de fondo (`SwingWorker`) para asegurar que la aplicación nunca se congele durante las invocaciones remotas de red.

1. Cliente Conversor con GUI (`ClienteConversorGUI.java`):

Presenta un formulario limpio y moderno para ingresar la temperatura y seleccionar el sentido de la conversión (Celsius a Fahrenheit o viceversa) a través de un menú desplegable, reflejando el estado de conexión con el endpoint SOAP en la barra inferior.

```
package Ej_propuestos.ej_01;

import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import javax.swing.*;
import javax.swing.border.EmptyBorder;

public class ClienteConversorGUI extends JFrame {
    private JTextField txtTemperatura;
    private JComboBox<String> comboConversion;
    private JLabel lblResultado;
    private JButton btnConvertir;
    private ConversorSOAP conversor;
    private JLabel lblStatus;

    public ClienteConversorGUI() {
        // Establecer Look and Feel nativo
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            // Ignorar y usar predeterminado
        }

        setTitle("Conversor de Temperatura SOAP");
        setSize(450, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // Panel Principal con padding
        JPanel mainPanel = new JPanel();
        mainPanel.setLayout(new BorderLayout(10, 10));
        mainPanel.setBorder(new EmptyBorder(20, 20, 20, 20));
        mainPanel.setBackground(new Color(245, 247, 250));
        setContentPane(mainPanel);

        // Encabezado
        JLabel lblTitulo = new JLabel("Conversor de Temperatura SOAP", JLabel.CENTER);
        lblTitulo.setFont(new Font("Segoe UI", Font.BOLD, 18));
        lblTitulo.setForeground(new Color(33, 37, 41));
        mainPanel.add(lblTitulo, BorderLayout.NORTH);

        // Panel Central (Formulario)
        JPanel formPanel = new JPanel(new GridBagLayout());
        formPanel.setBackground(new Color(245, 247, 250));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 10, 10, 10);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Fila 1: Temperatura
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.3;
        JLabel lblTemp = new JLabel("Temperatura:");
        lblTemp.setFont(new Font("Segoe UI", Font.PLAIN, 14));
```

```
formPanel.add(lblTemp, gbc);

gbc.gridx = 1; gbc.gridy = 0; gbc.weightx = 0.7;
txtTemperatura = new JTextField();
txtTemperatura.setFont(new Font("Segoe UI", Font.PLAIN, 14));
formPanel.add(txtTemperatura, gbc);

// Fila 2: Tipo de Conversion
gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.3;
JLabel lblTipo = new JLabel("Conversion:");
lblTipo.setFont(new Font("Segoe UI", Font.PLAIN, 14));
formPanel.add(lblTipo, gbc);

gbc.gridx = 1; gbc.gridy = 1; gbc.weightx = 0.7;
String[] opciones = {"Celsius a Fahrenheit (°C -> °F)", "Fahrenheit a Celsius (°F -> °C)"};
comboConversion = new JComboBox<>(opciones);
comboConversion.setFont(new Font("Segoe UI", Font.PLAIN, 14));
formPanel.add(comboConversion, gbc);

// Fila 3: Boton
gbc.gridx = 0; gbc.gridy = 2; gbc.gridwidth = 2; gbc.weightx = 1.0;
btnConvertir = new JButton("Convertir Remotamente");
btnConvertir.setFont(new Font("Segoe UI", Font.BOLD, 14));
btnConvertir.setBackground(new Color(16, 185, 129));
btnConvertir.setForeground(Color.WHITE);
btnConvertir.setFocusPainted(false);
formPanel.add(btnConvertir, gbc);

mainPanel.add(formPanel, BorderLayout.CENTER);

// Panel Inferior (Resultado y Estado)
JPanel bottomPanel = new JPanel(new BorderLayout(5, 5));
bottomPanel.setBackground(new Color(245, 247, 250));

lblResultado = new JLabel("Resultado: Esperando entrada...", JLabel.CENTER);
lblResultado.setFont(new Font("Segoe UI", Font.BOLD, 16));
lblResultado.setForeground(new Color(37, 99, 235));
lblResultado.setBorder(new EmptyBorder(10, 0, 5, 0));
bottomPanel.add(lblResultado, BorderLayout.NORTH);

lblStatus = new JLabel("Iniciando conexion con SOAP...", JLabel.LEFT);
lblStatus.setFont(new Font("Segoe UI", Font.ITALIC, 11));
lblStatus.setForeground(Color.GRAY);
bottomPanel.add(lblStatus, BorderLayout.SOUTH);

mainPanel.add(bottomPanel, BorderLayout.SOUTH);

// Accion del boton
btnConvertir.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        realizarConversion();
    }
});

// Inicializar el cliente SOAP en un hilo separado
```

```
        conectarSOAP();
    }

    private void conectarSOAP() {
        SwingWorker<Void, Void> worker = new SwingWorker<Void, Void>() {
            @Override
            protected Void doInBackground() throws Exception {
                try {
                    URL url = new URL("http://localhost:8081/conversor?wsdl");
                    QName qname = new QName("http://conversor.soap/", "ConversorSOAPService");
                    Service service = Service.create(url, qname);
                    conversor = service.getPort(ConversorSOAP.class);
                } catch (Exception ex) {
                    throw new RuntimeException("No se pudo conectar al servidor SOAP. Asegurese de que el Publicador este ejecutandose.");
                }
                return null;
            }

            @Override
            protected void done() {
                try {
                    get();
                    lblStatus.setText("Conexion SOAP activa en http://localhost:8081/conversor");
                    lblStatus.setForeground(new Color(16, 185, 129));
                } catch (Exception e) {
                    lblStatus.setText("Error: " + e.getCause().getMessage());
                    lblStatus.setForeground(Color.RED);
                    JOptionPane.showMessageDialog(ClienteConversorGUI.this,
                        e.getCause().getMessage(),
                        "Error de Conexion SOAP",
                        JOptionPane.ERROR_MESSAGE);
                }
            }
        };
        worker.execute();
    }

    private void realizarConversion() {
        if (conversor == null) {
            JOptionPane.showMessageDialog(this,
                "El cliente SOAP no esta conectado. Reintentando conexion...",
                "Error",
                JOptionPane.WARNING_MESSAGE);
            conectarSOAP();
            return;
        }

        String inputStr = txtTemperatura.getText().trim();
        if (inputStr.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Por favor ingrese un valor de temperatura.",
                "Entrada Vacía",
                JOptionPane.WARNING_MESSAGE);
            return;
        }
    }
```

```
try {
    final double tempInput = Double.parseDouble(inputStr);
    final int index = comboConversion.getSelectedIndex();

    btnConvertir.setEnabled(false);
    btnConvertir.setText("Procesando...");
    lblResultado.setText("Enviando petición SOAP...");

    SwingWorker<Double, Void> worker = new SwingWorker<Double, Void>() {
        @Override
        protected Double doInBackground() throws Exception {
            if (index == 0) {
                return conversor.cToF(tempInput);
            } else {
                return conversor.fToC(tempInput);
            }
        }

        @Override
        protected void done() {
            btnConvertir.setEnabled(true);
            btnConvertir.setText("Convertir Remotamente");
            try {
                double resultado = get();
                if (index == 0) {
                    lblResultado.setText(String.format("Resultado: %.2f °F", resultado));
                } else {
                    lblResultado.setText(String.format("Resultado: %.2f °C", resultado));
                }
            } catch (Exception e) {
                lblResultado.setText("Error al realizar la conversion.");
                JOptionPane.showMessageDialog(ClienteConversorGUI.this,
                    "Error al invocar el servicio SOAP: " + e.getMessage(),
                    "Error de Servicio",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    };
    worker.execute();

} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this,
        "Por favor ingrese un numero decimal valido.",
        "Error de Formato",
        JOptionPane.ERROR_MESSAGE);
}

}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new ClienteConversorGUI().setVisible(true);
        }
    });
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

```
}
}
```

1.1 Compilación y Ejecución de Conversor con GUI:

1. Asegúrese de que los publicadores de los servicios estén activos en segundo plano o terminales dedicadas.
2. Compilación de todos los archivos .java en el directorio de código compartido:

```
javac -d . codigo_compartir/Ej_propuestos/ej_01/*.java
```

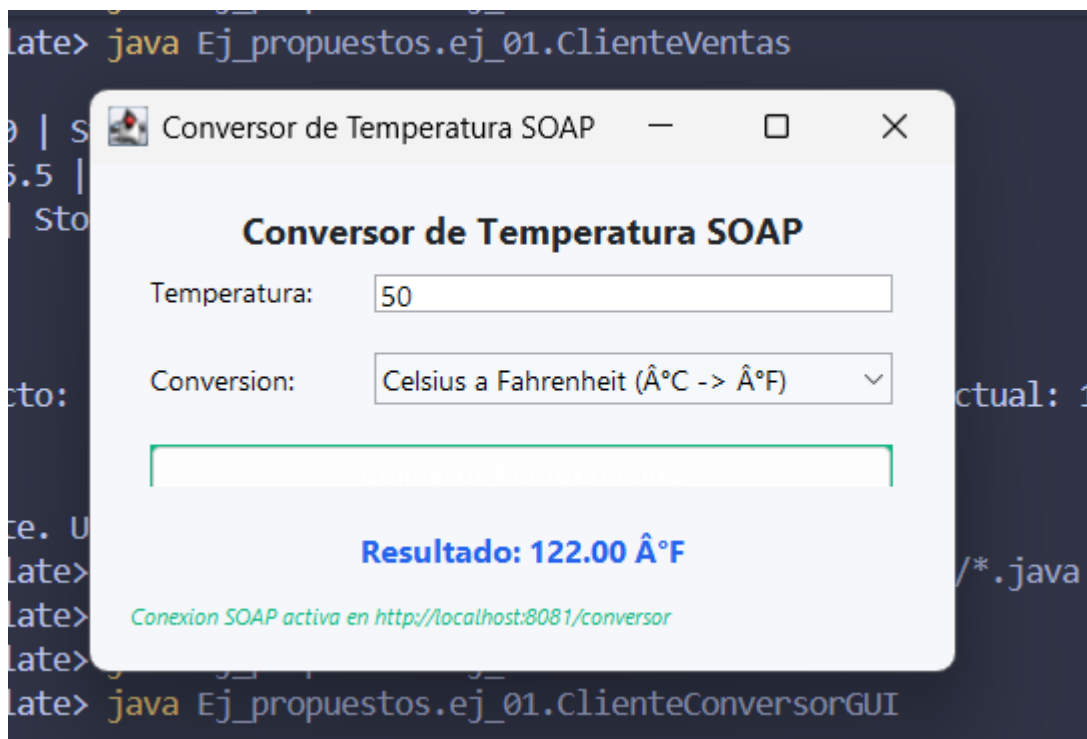
3. Ejecución del servicio

```
java Ej_propuestos.ej_01.Publicador
```

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_propuestos.ej_01.Publicador
Servicio SOAP de Conversor de Temperatura activo en: http://localhost:8081/conversor
```

3. Ejecución de la interfaz gráfica del Conversor:

```
java Ej_propuestos.ej_01.ClienteConversorGUI
```



2. Cliente de Ventas con GUI (ClienteVentasGUI.java):

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

Muestra el catálogo de productos disponibles en una tabla dinámica (JTable) que realiza una consulta inicial mediante obtenerProductos(). Al hacer clic sobre cualquier fila, el ID del producto se autocompleta automáticamente en el panel inferior, permitiendo ingresar la cantidad deseada y realizar compras instantáneas que decrementan el stock en el servidor en tiempo real.

```
package Ej_propuestos.ej_01;

import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;

public class ClienteVentasGUI extends JFrame {
    private JTable tablaProductos;
    private DefaultTableModel modeloTabla;
    private JTextField txtIdProducto;
    private JTextField txtCantidad;
    private JButton btnComprar;
    private JButton btnActualizar;
    private VentasSOAP servicioVentas;
    private JLabel lblStatus;

    public ClienteVentasGUI() {
        // Establecer Look and Feel nativo
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            // Ignorar
        }

        setTitle("Sistema de Ventas en Linea SOAP");
        setSize(650, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // Panel Principal
        JPanel mainPanel = new JPanel(new BorderLayout(15, 15));
        mainPanel.setBorder(new EmptyBorder(20, 20, 20, 20));
        mainPanel.setBackground(new Color(240, 244, 248));
        setContentPane(mainPanel);

        // Cabecera
        JPanel headerPanel = new JPanel(new BorderLayout());
        headerPanel.setBackground(new Color(240, 244, 248));
        JLabel lblTitulo = new JLabel("Catalogo de Productos - Cliente SOAP", JLabel.LEFT);
        lblTitulo.setFont(new Font("Segoe UI", Font.BOLD, 20));
        lblTitulo.setForeground(new Color(30, 41, 59));
        headerPanel.add(lblTitulo, BorderLayout.WEST);
    }
}
```

```
btnActualizar = new JButton("Actualizar Catalogo");
btnActualizar.setFont(new Font("Segoe UI", Font.BOLD, 12));
btnActualizar.setBackground(new Color(59, 130, 246));
btnActualizar.setForeground(Color.BLACK);
btnActualizar.setFocusPainted(false);
headerPanel.add(btnActualizar, BorderLayout.EAST);

mainPanel.add(headerPanel, BorderLayout.NORTH);

// Tabla de Productos en ScrollPane
String[] columnas = {"ID Producto", "Nombre del Producto", "Precio Unitario ($)", "Stock Disponible"};
modeloTabla = new DefaultTableModel(columnas, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};

tablaProductos = new JTable(modeloTabla);
tablaProductos.setFont(new Font("Segoe UI", Font.PLAIN, 13));
tablaProductos.setRowHeight(24);
tablaProductos.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 13));

JScrollPane scrollPane = new JScrollPane(tablaProductos);
mainPanel.add(scrollPane, BorderLayout.CENTER);

// Panel de Compra (Formulario)
JPanel compraPanel = new JPanel(new GridBagLayout());
compraPanel.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createEtchedBorder(),
    "Formulario de Compra",
    0, 0,
    new Font("Segoe UI", Font.BOLD, 14),
    new Color(30, 41, 59)
));
compraPanel.setBackground(new Color(240, 244, 248));

GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(8, 8, 8, 8);
gbc.fill = GridBagConstraints.HORIZONTAL;

// Fila 1: ID Producto
gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.2;
JLabel lblId = new JLabel("ID Producto seleccionado:");
lblId.setFont(new Font("Segoe UI", Font.PLAIN, 13));
compraPanel.add(lblId, gbc);

gbc.gridx = 1; gbc.gridy = 0; gbc.weightx = 0.3;
txtIdProducto = new JTextField();
txtIdProducto.setFont(new Font("Segoe UI", Font.PLAIN, 13));
compraPanel.add(txtIdProducto, gbc);

// Fila 2: Cantidad
gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.2;
```

```
JLabel lblCantidad = new JLabel("Cantidad a comprar:");
lblCantidad.setFont(new Font("Segoe UI", Font.PLAIN, 13));
compraPanel.add(lblCantidad, gbc);

gbc.gridx = 1; gbc.gridy = 1; gbc.weightx = 0.3;
txtCantidad = new JTextField();
txtCantidad.setFont(new Font("Segoe UI", Font.PLAIN, 13));
compraPanel.add(txtCantidad, gbc);

// Boton Comprar
gbc.gridx = 2; gbc.gridy = 0; gbc.gridheight = 2; gbc.weightx = 0.5;
gbc.fill = GridBagConstraints.BOTH;
btnComprar = new JButton("Realizar Compra SOAP");
btnComprar.setFont(new Font("Segoe UI", Font.BOLD, 14));
btnComprar.setBackground(new Color(16, 185, 129));
btnComprar.setForeground(Color.BLACK);
btnComprar.setFocusPainted(false);
compraPanel.add(btnComprar, gbc);

mainPanel.add(compraPanel, BorderLayout.SOUTH);

// Barra de Estado
lblStatus = new JLabel("Estableciendo conexion con servicio SOAP...", JLabel.LEFT);
lblStatus.setFont(new Font("Segoe UI", Font.ITALIC, 11));
lblStatus.setForeground(Color.GRAY);

JPanel statusPanel = new JPanel(new BorderLayout());
statusPanel.setBorder(new EmptyBorder(5, 5, 0, 5));
statusPanel.setBackground(new Color(240, 244, 248));
statusPanel.add(lblStatus, BorderLayout.CENTER);

// Vamos a reestructurar el layout del main panel para meter la barra de estado abajo
JPanel containerPanel = new JPanel(new BorderLayout());
containerPanel.add(mainPanel, BorderLayout.CENTER);
containerPanel.add(statusPanel, BorderLayout.SOUTH);
setContentPane(containerPanel);

// Escuchador para seleccionar productos haciendo click en la tabla
tablaProductos.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        int filaSeleccionada = tablaProductos.getSelectedRow();
        if (filaSeleccionada != -1) {
            Object idValue = modeloTabla.getValueAt(filaSeleccionada, 0);
            txtIdProducto.setText(idValue.toString());
            txtCantidad.requestFocus();
        }
    }
});

// Eventos
btnActualizar.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        cargarProductos();
    }
});
```

```
});

btnComprar.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ejecutarCompra();
    }
});

// Conectar al servicio
conectarSOAP();
}

private void conectarSOAP() {
    SwingWorker<Void, Void> worker = new SwingWorker<Void, Void>() {
        @Override
        protected Void doInBackground() throws Exception {
            try {
                URL url = new URL("http://localhost:8082/ventas?wsdl");
                QName qname = new QName("http://ventas.soap/", "VentasSOAPService");
                Service service = Service.create(url, qname);
                servicioVentas = service.getPort(VentasSOAP.class);
            } catch (Exception ex) {
                throw new RuntimeException("No se pudo conectar al servidor de Ventas. Verifique que PublicadorVentas este corriendo.");
            }
            return null;
        }

        @Override
        protected void done() {
            try {
                get();
                lblStatus.setText("Conexion SOAP de Ventas establecida en http://localhost:8082/ventas");
                lblStatus.setForeground(new Color(16, 185, 129));
                cargarProductos();
            } catch (Exception e) {
                lblStatus.setText("Error: " + e.getCause().getMessage());
                lblStatus.setForeground(Color.RED);
                JOptionPane.showMessageDialog(ClienteVentasGUI.this,
                    e.getCause().getMessage(),
                    "Error de Conexion Ventas SOAP",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    };
    worker.execute();
}

private void cargarProductos() {
    if (servicioVentas == null) {
        return;
    }

    btnActualizar.setEnabled(false);
}
```

```
lblStatus.setText("Cargando catalogo desde el servicio SOAP...");

SwingWorker<Producto[], Void> worker = new SwingWorker<Producto[], Void>() {
    @Override
    protected Producto[] doInBackground() throws Exception {
        return servicioVentas.obtenerProductos();
    }

    @Override
    protected void done() {
        btnActualizar.setEnabled(true);
        try {
            Producto[] productos = get();
            modeloTabla.setRowCount(0);
            for (Producto p : productos) {
                modeloTabla.addRow(new Object[]{p.getId(), p.getNombre(), p.getPrecio(),
p.getStock()});
            }
            lblStatus.setText("Catalogo actualizado. " + productos.length + " productos
cargados.");
            lblStatus.setForeground(new Color(16, 185, 129));
        } catch (Exception e) {
            lblStatus.setText("Error al cargar catalogo.");
            lblStatus.setForeground(Color.RED);
            JOptionPane.showMessageDialog(ClienteVentasGUI.this,
                "Error al listar productos: " + e.getMessage(),
                "Error SOAP",
                JOptionPane.ERROR_MESSAGE);
        }
    }
};
worker.execute();
}

private void ejecutarCompra() {
    if (servicioVentas == null) {
        JOptionPane.showMessageDialog(this, "El servicio SOAP no esta conectado.", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    String idStr = txtIdProducto.getText().trim();
    String cantStr = txtCantidad.getText().trim();

    if (idStr.isEmpty() || cantStr.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Ingrese ID de Producto y Cantidad a comprar.",
"Entrada Invalida", JOptionPane.WARNING_MESSAGE);
        return;
    }

    try {
        final int idProducto = Integer.parseInt(idStr);
        final int cantidad = Integer.parseInt(cantStr);

        btnComprar.setEnabled(false);
        btnComprar.setText("Procesando Venta...");
    }
```

```
SwingWorker<String, Void> worker = new SwingWorker<String, Void>() {
    @Override
    protected String doInBackground() throws Exception {
        return servicioVentas.realizarVenta(idProducto, cantidad);
    }

    @Override
    protected void done() {
        btnComprar.setEnabled(true);
        btnComprar.setText("Realizar Compra SOAP");
        try {
            String resultado = get();
            JOptionPane.showMessageDialog(ClienteVentasGUI.this,
                resultado,
                "Resultado de la Operacion",
                resultado.startsWith("Error") ? JOptionPane.ERROR_MESSAGE :
JOptionPane.INFORMATION_MESSAGE);

            // Recargar catalogo para actualizar existencias
            cargarProductos();
            txtCantidad.setText("");
        } catch (Exception e) {
            JOptionPane.showMessageDialog(ClienteVentasGUI.this,
                "Fallo al contactar servicio de venta: " + e.getMessage(),
                "Error de Conexion",
                JOptionPane.ERROR_MESSAGE);
        }
    }
};
worker.execute();

} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Los campos de ID y Cantidad deben ser enteros
validos.", "Error de Formato", JOptionPane.ERROR_MESSAGE);
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new ClienteVentasGUI().setVisible(true);
        }
    });
}
}
```

1.2 Compilación y Ejecución de Clientes GUI (Java):

1. Ejecución del servicio:

```
java Ej_propuestos.ej_01.PublicadorVentas
```

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> java Ej_propuestos.ej_01.PublicadorVentas
Servicio SOAP de Ventas activo en: http://localhost:8082/ventas
```

2. Ejecución de la interfaz gráfica del Sistema de Ventas en línea:

```
java Ej_propuestos.ej_01.ClienteVentasGUI
```

Catalogo de Productos - Cliente SOAP Actualizar Catalogo

ID Producto	Nombre del Producto	Precio Unitario (\$)	Stock Disponible
101	Laptop Gamer	1200.0	5
102	Teclado Mecanico	85.5	20
103	Mouse Optico	35.0	15

Formulario de Compra

ID Producto seleccionado:

Cantidad a comprar:

Realizar Compra SOAP

Catalogo actualizado. 3 productos cargados.

Venta exitosa. ID Producto: 102. Cantidad: 3. Total cobrado: \$256.5. Stock actualizado.

3. Una aplicación con SOAP services: Transferencias interbancarias

Cuando un cliente del Banco A solicita enviar dinero al Banco B, el sistema genera un mensaje SOAP en formato XML con los datos de la operación (monto, cuenta origen y destino). Este mensaje viaja por HTTPS y llega al servicio SOAP del Banco B, que lo valida y responde con otro XML confirmando la transacción. Gracias a SOAP, ambos bancos pueden comunicarse de manera segura y estandarizada, incluso si sus plataformas están desarrolladas en lenguajes distintos

Un caso concreto es el del BCP, que utiliza SOAP services para sus procesos de transferencias interbancarias y pagos en línea, el sistema del BCP genera un mensaje SOAP en XML con los datos de la operación (monto, cuenta origen y destino). Ese mensaje viaja por HTTPS hacia la cámara de compensación interbancaria (CCE),

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 24

donde se valida y se reenvía al banco receptor. SOAP asegura que, aunque los sistemas estén desarrollados en plataformas distintas (Java en un banco, .NET en otro), la comunicación sea segura y estandarizada.

EJERCICIO 2: Cliente SOAP con Python y Consumo desde Navegador

Este ejercicio evalúa la interoperabilidad heterogénea consumiendo servicios SOAP web desde Python con la biblioteca zeep, e introduce un análisis práctico del consumo desde navegadores web mediante una aplicación HTML + JS.

A. Flujo de Funcionamiento del Cliente Python

1. El script `cliente_calculator.py` utiliza la clase `Client` de la biblioteca `zeep` apuntando al WSDL de una calculadora pública real (<http://www.dneonline.com/calculator.asmx?WSDL>).
2. Al ejecutarse, `zeep` descarga el WSDL y expone dinámicamente las operaciones matemáticas definidas en el contrato remoto como métodos de Python (`Add`, `Subtract`, etc.).
3. Cuando llamamos a `client.service.Add(5, 8)`, la biblioteca se encarga de empaquetar los parámetros en un sobre SOAP XML estructurado, realiza la llamada POST HTTP al servidor, recibe e interpreta el sobre de respuesta e imprime el resultado final (13).

Cliente SOAP en Python (`cliente_calculator.py`):

```
# Cliente SOAP en Python para consumir el servicio de calculadora de DneOnline.
import sys
try:
    from zeep import Client
except ImportError:
    print("Error: Se requiere la libreria zeep para ejecutar este script.")
    print("Instalela usando: pip install zeep")
    sys.exit(1)

def main():
    # URL del WSDL del servicio web de calculadora
    wsdl_url = 'http://www.dneonline.com/calculator.asmx?WSDL'

    print("Estableciendo conexion con el servicio SOAP: " + wsdl_url)
    try:
        # Creacion del cliente SOAP con zeep
        client = Client(wsdl_url)

        # Valores de prueba
        num1 = 5
        num2 = 8

        print("\n--- Ejecutando operaciones de prueba ---")

        # Operacion de Suma (Add)
        resultado_suma = client.service.Add(num1, num2)
        print("Suma: " + str(num1) + " + " + str(num2) + " = " + str(resultado_suma))

        # Operacion de Resta (Subtract)
        resultado_resta = client.service.Subtract(num1, num2)
        print("Resta: " + str(num1) + " - " + str(num2) + " = " + str(resultado_resta))
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 25

```

# Operacion de Multiplicacion (Multiply)
resultado_mult = client.service.Multiply(num1, num2)
print("Multiplicacion: " + str(num1) + " * " + str(num2) + " = " + str(resultado_mult))

# Operacion de Division (Divide)
resultado_div = client.service.Divide(num1, num2)
print("Division: " + str(num1) + " / " + str(num2) + " = " + str(resultado_div))

print("\nPrueba de consumo completada con éxito.")

except Exception as e:
    print("Ocurrió un error al interactuar con el servicio SOAP:")
    print(str(e))

if __name__ == '__main__':
    main()

```

B. Consumo de SOAP desde Navegador con Proxy CORS

Para lograr un consumo real desde el navegador sin bloqueos de CORS, se implementó un **proxy local en Python** que actúa como pasarela. El navegador envía un JSON con la operación y los operandos; el proxy invoca el servicio SOAP real (DneOnline) usando zeep y devuelve un JSON limpio al frontend. De esta forma el navegador nunca construye XML y la llamada funciona sin restricciones de origen cruzado.

Servidor Proxy con CORS (soap_proxy_cors.py):

```

# Proxy CORS para consumir el servicio SOAP de DneOnline desde navegador.
# Expone un endpoint REST local y llama a SOAP con zeep.

import json
from http.server import BaseHTTPRequestHandler, HTTPServer
from socketserver import ThreadingMixIn

try:
    from zeep import Client
except ImportError:
    raise SystemExit("Error: Instale zeep con: pip install zeep")

WSDL_URL = "http://www.dneonline.com/calculator.asmx?WSDL"
HOST = "localhost"
PORT = 8090

SOAP_CLIENT = Client(WSDL_URL)

class ThreadingHTTPServer(ThreadingMixIn, HTTPServer):
    daemon_threads = True

def ejecutar_operacion(operacion, a, b):
    if operacion == "Add":
        return SOAP_CLIENT.service.Add(a, b), "SOAP: Add"
    if operacion == "Subtract":

```

```
        return SOAP_CLIENT.service.Subtract(a, b), "SOAP: Subtract"
    if operacion == "Multiply":
        return SOAP_CLIENT.service.Multiply(a, b), "SOAP: Multiply"
    if operacion == "Divide":
        if b == 0:
            raise ValueError("Division por cero no permitida.")
        return SOAP_CLIENT.service.Divide(a, b), "SOAP: Divide"
    if operacion == "Modulo":
        if b == 0:
            raise ValueError("Modulo por cero no permitido.")
        cociente = SOAP_CLIENT.service.Divide(a, b)
        producto = SOAP_CLIENT.service.Multiply(b, cociente)
        residuo = SOAP_CLIENT.service.Subtract(a, producto)
        return residuo, "SOAP: Divide + Multiply + Subtract"
    raise ValueError("Operacion no soportada.")

class SoapProxyHandler(BaseHTTPRequestHandler):
    def _send_headers(self, status=200, content_type="application/json"):
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Access-Control-Allow-Origin", "*")
        self.send_header("Access-Control-Allow-Methods", "POST, OPTIONS")
        self.send_header("Access-Control-Allow-Headers", "Content-Type")
        self.send_header("Access-Control-Max-Age", "86400")
        self.end_headers()

    def do_OPTIONS(self):
        self._send_headers(204)

    def do_POST(self):
        if self.path != "/api/calculadora":
            self._send_headers(404)
            self.wfile.write(json.dumps({"error": "Ruta no encontrada"}).encode("utf-8"))
            return

        try:
            length = int(self.headers.get("Content-Length", "0"))
            body = self.rfile.read(length).decode("utf-8")
            payload = json.loads(body)
            operacion = payload.get("operacion")
            a = int(payload.get("a"))
            b = int(payload.get("b"))
        except Exception as exc:
            self._send_headers(400)
            self.wfile.write(
                json.dumps({"error": f"Solicitud invalida: {exc}"}).encode("utf-8")
            )
            return

        try:
            resultado, detalle = ejecutar_operacion(operacion, a, b)
            self._send_headers(200)
            self.wfile.write(
                json.dumps(
                    {
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 27

```

        "ok": True,
        "operacion": operacion,
        "a": a,
        "b": b,
        "resultado": resultado,
        "wsdl": WSDL_URL,
        "detalle": detalle,
    }
    ).encode("utf-8")
)
except Exception as exc:
    self._send_headers(500)
    self.wfile.write(json.dumps({"error": str(exc)}).encode("utf-8"))

def log_message(self, format, *args):
    return

def main():
    server = ThreadingHTTPServer((HOST, PORT), SoapProxyHandler)
    print(f"Proxy CORS activo en http://{HOST}:{PORT}/api/calculadora")
    print(f"Usando WSDL remoto: {WSDL_URL}")
    server.serve_forever()

if __name__ == "__main__":
    main()

```

C. Análisis de Limitaciones del Consumo Directo de SOAP en Navegador

El consumo directo sigue presentando restricciones técnicas importantes:

1. **Bloqueo por CORS (Cross-Origin Resource Sharing):** Los servidores SOAP tradicionales no retornan Access-Control-Allow-Origin, por lo que el navegador bloquea la respuesta. El proxy soluciona esto agregando CORS y actuando como mismo origen.
2. **Complejidad de Generación y Parsing Manual:** SOAP exige construir sobres XML y parsear respuestas XML, lo que eleva la complejidad del frontend frente a JSON.
3. **Riesgos de Seguridad (Exposición de Credenciales):** Incluir WS-Security en JavaScript expone credenciales en el cliente.
4. **Sobrecarga de Datos (Protocol Overhead):** XML con namespaces es más pesado que JSON y afecta el rendimiento en el navegador.

D. Flujo de Ejecución Correcto (Sin Tkinter, Todo en el Navegador)

1. El usuario interactúa con cliente_web.html y selecciona operaciones **Sumar, Restar, Multiplicar, Dividir y Módulo**.
2. El frontend envía un JSON vía fetch al proxy `http://localhost:8090/api/calculadora`.
3. El proxy invoca el servicio SOAP de DneOnline con zeep (y para **Módulo** compone el resultado usando operaciones SOAP base).
4. El proxy responde con JSON y el navegador muestra el resultado sin errores de CORS.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 28

Observación y reflexión: Con el proxy CORS activo, el navegador deja de fallar por políticas de origen y el consumo SOAP se vuelve transparente para el usuario. El patrón de pasarela separa la complejidad XML del frontend y habilita una interfaz web moderna sin perder interoperabilidad con el servicio SOAP.

Compilacion y ejecucion - Proxy CORS + Interfaz Web

1. Instalación de la biblioteca zeep necesaria para el proxy:

```
pip install zeep
```

```
(env) PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> python codigo_compartir/Ej_propuestos/ej_02/cliente_calculator.py
Estableciendo conexion con el servicio SOAP: http://www.dneonline.com/calculator.asmx?WSDL

--- Ejecutando operaciones de prueba ---
Suma: 5 + 8 = 13
Resta: 5 - 8 = -3
Multiplicacion: 5 * 8 = 40
Division: 5 / 8 = 1

Prueba de consumo completada con exito.
```

2. Ejecución del proxy CORS:

```
python codigo_compartir/Ej_propuestos/ej_02/soap_proxy_cors.py
```

```
PS C:\Users\fabia\Documents\UNSA\2026-I\SD\template> python codigo_compartir/Ej_propuestos/ej_02/soap_proxy_cors.py
Proxy CORS activo en http://localhost:8090/api/calculadora
Usando WSDL remoto: http://www.dneonline.com/calculator.asmx?WSDL
```

3. Abrir codigo_compartir/Ej_propuestos/ej_02/cliente_web.html en el navegador.
4. Ingresar los operandos y seleccionar la operación; el resultado se mostrará de forma inmediata en la consola de la interfaz.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 29

Cliente SOAP Web con Proxy CORS

Ejecutar Petición SOAP

API Proxy (URL):

`http://localhost:8090/api/calculadora`

Operando A: Operando B:

Operación:

Módulo (Modulo) ▼

Calcular

Payload JSON Generado:

```
{
  "operacion": "Modulo",
  "a": 10,
  "b": 20
}
```

Resultado de la Invocación

Exito

Conexion exitosa!

Operacion: Modulo
Operandos: 50 y 20
Resultado: 10
WSDL: <http://www.dneonline.com/calculator.asmx?WSDL>
Detalle: SOAP: Divide + Multiply + Subtract

Flujo Correcto y Observaciones

- ✦ **CORS resuelto:** El navegador llama al proxy local con CORS habilitado y el proxy consume SOAP sin restricciones.
- ✦ **XML oculto:** El frontend trabaja con JSON; el sobre SOAP se arma en el servidor con `zeep`.
- ✦ **Respuesta limpia:** El proxy devuelve un JSON con el resultado listo para mostrar.
- ✦ **Operaciones ampliadas:** Se incluyen Sumar, Restar, Multiplicar, Dividir y Módulo.

II. CUESTIONARIO

1. ¿Puedo utilizar un componente JavaBeans para implementar un servicio web utilizando la invocación de SOAP sobre JMS (Java Message Service)?

Sí, pero no cualquier JavaBean: se utiliza un Message-Driven Bean (MDB) Un JavaBean es una clase Java reutilizable que encapsula datos o lógica, y en el ámbito empresarial se usa como Enterprise JavaBean (EJB), gestionado por un servidor de aplicaciones. Cuando se combinan, el cliente SOAP no envía su mensaje por HTTP, sino que lo serializa en XML y lo transmite como un mensaje JMS. El servidor de aplicaciones recibe ese mensaje en un Message-Driven Bean (MDB), que está diseñado para consumir mensajes de forma asíncrona. La gran ventaja es que el servidor controla el ritmo de consumo: si está saturado, puede pausar la lectura de la cola, evitando caídas por sobrecarga. Esto convierte SOAP+JMS en una arquitectura de acoplamiento débil y resiliente, ideal para sistemas que deben absorber ráfagas de tráfico sin colapsar.

Esta arquitectura no es solo una solución funcional, sino que se alinea con los principios de diseño recomendados por Gregor Hohpe y Bobby Woolf - Enterprise Integration Patterns. Según los autores, el uso de componentes especializados como el Message-Driven Bean (MDB) es la forma correcta de implementar el patrón Message Endpoint. Este enfoque permite que la lógica de negocio permanezca 'agnóstica' al transporte, asegurando que el

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 30

sistema sea más fácil de mantener y escalar, tal como sugieren las mejores prácticas en sistemas de mensajería empresarial. [4]

2. ¿Cómo funciona la mensajería bidireccional con la implementación de SOAP y JMS? ¿Da soporte a varios clientes realizando solicitudes simultáneas?

SOAP normalmente funciona en modo request/response, mientras que JMS es intrínsecamente asíncrono. Para lograr bidireccionalidad se utiliza el patrón Request/Reply. El cliente envía un mensaje JMS indicando en el encabezado JMSReplyTo la cola donde espera la respuesta y asigna un JMSCorrelationID para identificar la petición. El servidor procesa el mensaje y responde en la cola indicada, reutilizando el mismo CorrelationID para que el cliente pueda asociar la respuesta con la petición original. En escenarios con múltiples clientes simultáneos, el broker JMS distribuye los mensajes entre varios consumidores (Competing Consumers), lo que permite balanceo automático y tolerancia a fallos. De esta forma, SOAP sobre JMS soporta concurrencia masiva y escalabilidad horizontal.

3. ¿Por qué SOAP sigue siendo utilizado en sistemas empresariales críticos pese al auge de REST?

Aunque REST se ha popularizado por su simplicidad y ligereza, SOAP ofrece garantías formales que son esenciales en entornos donde la fiabilidad es más importante que la velocidad. Por ejemplo, soporta transacciones distribuidas (WS-AtomicTransaction), que aseguran que una operación compleja se ejecute en modo “todo o nada”, algo vital en pagos interbancarios. Además, SOAP define contratos estrictos mediante WSDL, lo que garantiza que los servicios sean inmutables y que cualquier cambio rompa la compilación, evitando errores silenciosos. También incorpora protocolos de fiabilidad como WS-ReliableMessaging, que aseguran entrega, reintentos y orden de mensajes.

Un beneficio adicional y muy importante es su interoperabilidad: SOAP fue diseñado para que sistemas desarrollados en lenguajes distintos (Java, .NET, PHP, C++) puedan comunicarse sin ambigüedades. Gracias al uso de XML y WSDL, el contrato del servicio es independiente de la plataforma, lo que permite que un banco con sistemas en COBOL pueda integrarse con otro que usa Java, o que un sistema gubernamental en .NET pueda interoperar con un ERP en Oracle. Esta capacidad de conectar ecosistemas heterogéneos es uno de los motivos por los que SOAP sigue siendo la opción preferida en banca para procesos interbancarios, salud y sistemas de gobierno donde sobresale la heterogeneidad a relucir.

4. ¿Qué implicancias tiene el uso de XML en el rendimiento de sistemas distribuidos de alta concurrencia?

XML es más pesado que JSON, pero aporta **validación estricta** y seguridad avanzada. El impacto en rendimiento depende del parser utilizado: el modelo DOM es costoso porque carga todo el documento en memoria, mientras que StAX procesa en streaming con consumo constante de memoria, lo que lo hace más eficiente en sistemas de alta concurrencia. Además, XML permite **firmas digitales y cifrado granular** como soporte nativo, lo que significa que se puede proteger solo una parte del documento (ejemplo: número de cuenta) y dejar visible otra (ejemplo: ID de transacción). Aunque formatos binarios como Protobuf o Avro son más rápidos, XML sigue siendo preferido en entornos regulados porque ofrece trazabilidad, auditoría y seguridad, aspectos más importantes que la velocidad pura, como suele ser en bancos.

Como señalan los expertos en arquitecturas de software, la elección del formato de serialización debe equilibrar la eficiencia del parseo con las necesidades de auditoría y cumplimiento normativo [5]

5. ¿En qué escenarios arquitectónicos SOAP resulta más adecuado que REST? Justifique técnicamente.

SOAP resulta más adecuado en escenarios donde se necesita control total sobre el ciclo de vida del mensaje y la calidad del servicio (QoS). En arquitecturas de integración empresarial (ESB), buses como TIBCO o Mulesoft

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 31

trabajan nativamente con WSDL y protocolos SOAP. En entornos de seguridad avanzada, SOAP soporta estándares como WS-Security y la propagación de identidad mediante tokens SAML, lo que permite que un mensaje viaje por múltiples servidores sin perder la autenticación original. Además, en sistemas donde la garantía de entrega es crítica (órdenes médicas, transacciones bursátiles), SOAP ofrece WS-ReliableMessaging, que asegura que los mensajes lleguen y se procesen en orden, incluso ante fallos de red. En cambio, REST es ideal para aplicaciones web y microservicios ligeros, pero SOAP es insustituible en arquitecturas críticas, reguladas y de misión empresarial. Mientras REST simplifica la integración en microservicios, SOAP proporciona una “capa de transporte de misión crítica” que asegura la consistencia en transacciones distribuidas, un principio arquitectónico fundamental en sistemas de alta complejidad[6]

III. CONCLUSIONES

1. **Sobre el uso de SOAP** SOAP se emplea en sistemas críticos donde la seguridad, la fiabilidad y la interoperabilidad son indispensables. Su diseño basado en XML y contratos WSDL permite que aplicaciones escritas en distintos lenguajes y plataformas se comuniquen de manera estandarizada, lo que lo hace ideal en entornos heterogéneos como banca, salud y gobierno.
2. **Sobre sus limitaciones:** Las principales limitaciones son su complejidad y su peso. Los mensajes XML son más grandes y requieren más procesamiento que JSON, lo que impacta en el rendimiento. Además, su implementación exige mayor infraestructura y conocimientos especializados, lo que lo hace menos práctico para aplicaciones ágiles o servicios web ligeros.
3. **Sobre su potencial:** El potencial de SOAP radica en su robustez. Ofrece soporte nativo para transacciones distribuidas, seguridad avanzada (WS-Security), fiabilidad en la entrega (WS-ReliableMessaging) y contratos estrictos (WSDL). Estas características lo convierten en una herramienta insustituible cuando se necesita garantizar integridad, trazabilidad y cumplimiento normativo.
4. **Aprendizaje metacognitivo:** SOAP no debe verse como obsoleto frente a REST, sino como una solución especializada para escenarios donde la confiabilidad y la interoperabilidad son más importantes que la simplicidad. REST es excelente para aplicaciones modernas y microservicios ligeros, pero SOAP sigue siendo la opción preferida en arquitecturas críticas. La reflexión final es que SOAP y REST son complementarios: cada uno responde a necesidades distintas dentro del ecosistema de servicios distribuidos.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] A. S. Tanenbaum and M. Van Steen, **Distributed Systems: Principles and Paradigms**, 2nd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2006, pp. 235-260.
- [2] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, **Distributed Systems: Concepts and Design**, 5th ed. Boston, MA, USA: Addison-Wesley, 2011, pp. 605-620.
- [3] F. Cristian, “Probabilistic clock synchronization,” **Distributed Computing**, vol. 3, no. 3, pp. 146-158, 1989.
- [4] Hohpe, G., y Woolf, B. (2003). **Enterprise integration patterns: Designing, building, and deploying messaging solutions**. Addison-Wesley Professional.
- [5] R. E. Filman y T. E. Fuhrman, **Aspect-Oriented Software Development**. Addison-Wesley, 2005.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 32

- [6] T. Erl, **SOA: Principles of Service Design**. Prentice Hall, 2007